Code Standards

A Developer's Reference Guide to Writing Clean, Maintainable Code

1. Naming Conventions

Variables & Functions — camelCase

Use descriptive camelCase names for variables and functions. Names should reveal intent.

- Avoid single-letter names (except loop counters).
- Boolean variables should start with is, has, or can.
- Functions should be verbs: getUser(), calculateTotal().

```
// Bad let d = 30; let flag = true; // Good let sessionDurationDays = 30; let isLoggedIn = true; function fetchUserById(id) { ... }
```

Classes & Types — PascalCase

Classes, interfaces, enums, and type aliases use PascalCase.

- Class names should be nouns: UserService, OrderRepository.
- Interfaces often prefixed with I (IEmailSender) in typed languages.
- Enums use PascalCase for name and UPPER_SNAKE_CASE for members.

```
// Good class PaymentProcessor { ... } interface INotificationService { ... } enum OrderStatus { PENDING, CONFIRMED, SHIPPED }
```

Constants — UPPER_SNAKE_CASE

Global constants and configuration values use uppercase with underscores.

- Signals to developers that the value must not be mutated.
- Group related constants in a config or constants file.
- Avoid magic numbers — name them.

```
// Bad setTimeout(fn, 86400000); // Good const SECONDS_PER_DAY = 86_400; const MAX_RETRY_ATTEMPTS = 3;
```

Files & Folders — kebab-case

File and directory names use lowercase with hyphens for portability across OS.

- Avoids case-sensitivity issues between macOS and Linux.
- Component files may mirror their export: UserProfile.tsx → user-profile.tsx.
- Group by feature, not by type: /users/ over /controllers/.

```
# Good structure src/ users/ user-service.ts user-repository.ts user.model.ts
```

2. Code Formatting

Indentation & Spacing

Consistent indentation (2 or 4 spaces — never tabs mixed with spaces) improves readability across editors.

- Use a project-wide config: .editorconfig or Prettier.
- One blank line between logical blocks; two before top-level declarations.
- Max line length: 80–120 characters to avoid horizontal scrolling.

```
// .editorconfig root = true [*] indent_style = space indent_size = 2 max_line_length = 100
```

Braces & Brackets

Opening braces on the same line as the statement (K&R; style) is the most common convention.

- Always use braces — even for single-line if/else blocks.
- Avoids the 'goto fail' class of bugs.
- Auto-formatters (Prettier, Black, gofmt) enforce this automatically.

```
// Bad — braces omitted if (isValid) doSomething(); // Good if (isValid) { doSomething(); }
```

Linters & Formatters

Automated tools enforce style rules on save or in CI, removing subjective debates in code reviews.

- JavaScript/TypeScript: ESLint + Prettier.
- Python: Flake8 / Ruff + Black.
- Run formatters in a pre-commit hook (Husky, lint-staged).

```
# package.json scripts "lint": "eslint . --ext .ts", "format": "prettier --write .", "pre-commit": "lint-staged"
```

3. Commenting & Documentation

Write Self-Documenting Code First

Good code reads like prose. Comments should explain WHY, not WHAT — the code already shows what.

- Prefer clear names over explanatory comments.
- A comment that restates the code is noise.
- If you need a comment to explain what the code does, refactor first.

```
// Bad — comment restates code i++; // increment i // Good — comment explains WHY // Skip the first item; it's a placeholder header i++;
```

JSDoc / Docstrings for Public APIs

Public functions, classes, and modules must have structured documentation comments.

- Document parameters, return type, and exceptions.
- Enables IDE auto-complete and hover docs.
- Tools: JSDoc (JS/TS), docstring + Sphinx (Python), Javadoc (Java).

```
/** * Calculates compound interest. * @param principal - Initial amount * @param rate - Annual interest rate (0–1) * @param years - Investment duration * @returns Final amount after compound growth */ function compoundInterest(principal, rate, years) { ... }
```

TODO / FIXME Conventions

Use standardised tags for inline notes so they're searchable and trackable.

- TODO: feature or improvement to be added.
- FIXME: known bug that must be addressed.
- HACK / NOTE: temporary workaround with an explanation.
- Link to a ticket when possible: // TODO(#342): replace with cache layer

```
// TODO(#101): paginate this query when dataset > 10k rows const users = await db.query('SELECT * FROM users');
```

4. Error Handling

Never Swallow Errors

Silently catching exceptions hides bugs. Always log or re-throw errors with context.

- An empty catch block is almost always a bug.
- Log the error with stack trace and relevant context.
- Use structured logging (JSON) so errors are searchable in production.

```
// Bad try { parseConfig(file); } catch (e) {} // Good try { parseConfig(file); } catch (e) { logger.error('Failed to parse config', { file, error: e.message }); throw e; }
```

Use Custom Error Classes

Custom error types make error handling explicit and allow fine-grained catch logic.

- Extend the base Error class.
- Add status codes or error codes to distinguish types.
- Avoid using generic Error for all cases.

```
class NotFoundError extends Error { constructor(resource, id) { super(`${resource} with id ${id} not found`); this.statusCode = 404; this.name = 'NotFoundError'; } }
```

Fail Fast & Validate Early

Validate inputs at the boundary of your system (API layer, function entry) before processing begins.

- Reject bad input immediately — don't let it propagate deep.
- Use schema validation libraries: Zod, Joi, Pydantic.
- Return clear, actionable error messages to the caller.

```
import { z } from 'zod'; const UserSchema = z.object({ email: z.string().email(), age: z.number().min(18), }); const user = UserSchema.parse(req.body); // throws on invalid input
```

5. Code Structure & Organisation

DRY — Don't Repeat Yourself

Every piece of knowledge must have a single, authoritative representation in the codebase.

- Extract duplicated logic into shared utilities or base classes.
- But don't abstract prematurely — duplicate twice, then extract.
- WET code (Write Everything Twice) is sometimes okay initially.

```
// Bad — duplicated validation function createUser(email) { if (!email.includes('@')) throw... } function updateUser(email) { if (!email.includes('@')) throw... } // Good function validateEmail(email) { if (!email.includes('@')) throw... }
```

Function Size — Do One Thing

A function should do one thing and do it well. If you need 'and' to describe it, split it.

- Aim for functions under 20–30 lines.
- More than 3 parameters? Consider a config object.
- Deeply nested code (3+ levels) is a sign to extract.

```
// Bad — does too much function processOrder(order) { // validate, save, send email, update inventory... } // Good — split responsibilities validateOrder(order); await saveOrder(order); await notifyCustomer(order); await updateInventory(order);
```

Separation of Concerns (SoC)

Different responsibilities belong in different layers — presentation, business logic, and data access must not bleed into each other.

- Controllers handle HTTP; services handle logic; repositories handle data.
- Don't write SQL in your route handlers.
- Don't put business rules in your database triggers.

```
// Layered architecture router.post('/orders', orderController.create); // HTTP layer // orderController calls → orderService.createOrder() // Logic layer // orderService calls → orderRepository.save() // Data layer
```

6. Version Control Standards

Conventional Commits

Use a structured commit message format so history is machine-readable and changelog generation is automated.

- Format: type(scope): description
- Types: feat, fix, docs, style, refactor, test, chore.
- Enables tools like semantic-release, commitizen.

```
# Good commit messages feat(auth): add JWT refresh token rotation fix(orders): handle null discount code gracefully docs(readme): update local setup instructions refactor(users): extract email validation to utility
```

Branch Naming

Branch names should convey the purpose and link to a ticket where possible.

- Pattern: type/ticket-id-short-description.
- Keep it lowercase, hyphen-separated.
- Delete branches after merging to keep the repo clean.

```
# Good branch names feature/AUTH-42-oauth-google-login bugfix/ORDERS-88-null-discount-crash chore/update-node-18-to-20 hotfix/PROD-critical-payment-timeout
```

Pull Request Standards

PRs should be small, focused, and reviewed before merging to main.

- One PR = one concern. Large PRs slow reviews and hide bugs.
- Include: what changed, why, and how to test.
- Require at least one approving review + passing CI before merge.

```
## PR Description Template ### What changed? Added Google OAuth login flow. ### Why? Ticket AUTH-42 — users requested social login. ### How to test? 1. Click 'Login with Google' 2. Verify redirect and session creation.
```

7. Testing Standards

Test Naming — Describe Behaviour

Test names should read as specifications: given X, when Y, then Z.

- Names should fail descriptively so you know what broke.
- Group with describe() blocks by class or function.
- Avoid vague names like test1 or should work.

```
describe('calculateDiscount', () => { it('returns 10% off for premium users', () => { ... }); it('returns 0% for guests', () => { ... }); it('throws if discount rate exceeds 100', () => { ... }); });
```

AAA Pattern — Arrange, Act, Assert

Structure every test in three clear phases to make intent obvious.

- Arrange: set up data and mocks.
- Act: call the function under test.
- Assert: verify the outcome.

```
it('sends welcome email on user registration', async () => { // Arrange const emailSpy = jest.spyOn(emailService, 'send'); const newUser = { email: 'a@b.com', name: 'Ada' }; // Act await userService.register(newUser); // Assert expect(emailSpy).toHaveBeenCalledWith('a@b.com', 'Welcome!'); });
```

Test Coverage — Quality Over Quantity

Aim for meaningful coverage of critical paths, not 100% line coverage at the cost of test quality.

- Cover: happy path, edge cases, and error paths.
- 80% coverage on critical modules is a reasonable target.
- A passing test suite with bad assertions is worse than no tests.

```
# What to prioritise ✓ Core business logic (payment, auth, orders) ✓ Edge cases (empty input, max values, null) ✓ Error paths (DB failure, network timeout) ✗ Don't test framework code or trivial getters/setters
```